

C++ 编程入门

CSP-J 方向

变量状态追踪： 赋值与顺序执行

学段：七年级 课时：45 分钟 班级：24 人

本课只做两件事

读懂赋值 · 逐行追踪变量值

先算右侧，再写回左侧

旧值要保留

预测 → 揭示 → 状态表

核心概念

`=` 是单向箭头：先算右侧 → 再写回左侧

不是方程，不反向传播；左侧变量在右侧被求值时仍保持旧值。

```
1 int a = 2;
```

```
2 a = a + 3; ← 新值 5
```

```
3 cout << a;
```

① 读旧

$a = 2$

② 算右

$2+3=5$

③ 写回

$a = 5$

左侧旧值 → 右侧新值

读旧： $a = 2$

算右： $2 + 3 = 5$

$a_{旧}=2 \rightarrow a_{新}=5$

下一行读"新"5

心法 1

先算右，再写左

心法 2

不反向传播

心法 3

查状态表

单变量自更新：先读旧，再写新

```
1  int a = 2;
2  a = a + 3;
3  cout << a;
```

状态轨迹：2 → 5 → 输出 5

行	代码	● a	输出
1	int a = 2;	2	—
2	a = a + 3;	5	—
3	cout << a;	5	5

公布答案：5 · 口诀：右侧用旧值，写回是新值。

双变量顺序依赖：后写者能读到新值

```

1  int x = 4;
2  int y = x + 2;
3  x = y - 1;
4  cout << x;

```

依赖轨迹：x=4 → y=6 → x=5 → 输出5

行	代码	x	y	输出
1	int x = 4;	4	—	—
2	int y = x + 2;	4	6	—
3	x = y - 1;	5	6	—
4	cout << x;	5	6	5

公布答案：5 · 关键点：第 2 行的 x 是旧值 4；第 3 行的 y 已经是新值 6。

要交换，先借 t：临时变量保留旧值

三步顺序： `t = x` → `x = y` → `y = t`

```
1  int x = 2, y = 7;
2  int t = x;
3  x = y;
4  y = t;
5  cout << x << y;
```

行	代码	x	y	t	输出
1	<code>int x = 2, y = 7;</code>	2	7	—	—
2	<code>int t = x;</code>	2	7	2	—
3	<code>x = y;</code>	7	7	2	—
4	<code>y = t;</code>	7	2	2	—
5	<code>cout << x << y;</code>	7	2	2	72

视觉追踪：旧值 `x=2` → `t=2` → `x=7`；旧值 `y=7` → `y=2`。t 像备忘录，先保存再覆盖。

常见错：删除 t 直接 `x = y; y = x;` 会得到 77，因为 y 的旧值已被覆盖。

旧值混淆陷阱：`a = b` 之后再读 a

预测口诀：赋值是「快照」，不是「订阅」。

先预测：a = b; b = a + 2; 之后 a、b 分别是几？

```
1 int a = 3, b = 5;
2 a = b;
3 b = a + 2;
4 cout << a << b;
```

第 3 行读到的新 a 是 5，所以 b 变成 7；输出 57。

行	代码	a	b	输出
1	int a=3,b=5;	3	5	—
2	a = b;	3 → 5	5	—
3	b = a + 2;	5	5 → 7	—
4	cout << a << b;	5	7	57

链式自更新：x = x + 1; x = x * 3;

预测口诀：每一行的右侧都从「上一行写回的新值」出发。

先预测，再揭示：第三行右侧的 x 是 1 还是 2?

```
1  int x = 1;
2  x = x + 1;
3  x = x * 3;
4  cout << x;
```

行	代码	x	输出
1	int x = 1;	1	—
2	x = x + 1;	2	—
3	x = x * 3;	6	—
4	cout << x;	6	6

下一行

重置

公布答案

值快照：b 拿到的是当时 a 的值

预测口诀：b 是「快照」，不是「订阅」。

先预测，再揭示：a = 4 之后，b 会跟着变成 4 吗？

```
1  int a = 10;
2  int b = a;
3  a = 4;
4  cout << b;
```

行	代码	a	b	输出
1	int a = 10;	10	—	—
2	int b = a;	10	10	—
3	a = 4;	4	10	—
4	cout << b;	4	10	10

下一行

重置

公布答案

三条心法，回到一行一状态

本课所有题目都围绕这三条心法展开；下节课将进入「分支结构 if / else」。

心法 1

先算右侧，再写回左侧

``=`` 是单向箭头，从右指向左；不要把它当方程。

心法 2

旧值要保留

右侧求值时左侧仍是旧值；下一行才能读到上一行的新值。

心法 3

预测 → 揭示 → 状态表

每道题按这三步推进，把行级轨迹落到实处。

双变量互相依赖：第 2 行更新后，第 3 行要用新值

预测口诀：每行开始时所有变量都已更新到上一行最新值。

```
1  int p = 8, q = 2;
2  p = p - q;
3  q = p + q;
4  cout << p << q;
```

行	代码	p	q	输出
1	int p = 8, q = 2;	8	2	—
2	p = p - q;	6	2	—
3	q = p + q;	6	8	—
4	cout << p << q;	6	8	68

巩固 ② 表达式重赋值链

课后巩固 · 4 min

代码（连续自更新）：

```
// 第 1 行：先初始化
int n = 3;
n = n * n;    // ← 新值 9（读旧值 3）
n = n - 1;    // ← 新值 8（读上一步的 9）

cout << n;
```

状态表（每行开始前 n 已更新）

行	代码	n	输出
1	int n = 3;	3	—
2	n = n * n;	3 → 9	—
3	n = n - 1;	9 → 8	—
4	cout << n;	8	8

口诀：右读旧 · 左写新 · 行行接力。第 2 行 n*n 读旧 3 得 9；第 3 行再读这个新 9 得 8。

答案：8 · 第 3 行 n-1 中的 n 是上一步写回的新值 9